The `FilterSchema` base class contains a method called `.filter()`, which takes a `QuerySet` and filters it based on the instantiated values of the `FilterSchema` object. I tried to write the word *filter* twice more in that last sentence to make it clearer but couldn't sneak them in. Remember, as complicated as this sounds, it is just a set of query parameters. Ninja uses the query parameters to build a set of fields to perform a filter and provides a class that encapsulates this operation for you. The following listing has replacement code for the `venues()` endpoint.

Listing 11.10 Declare the fields that participate in a Ninja filter

```
# RiffMates/bands/api.py
...
from ninja import (Field, Router, ModelSchema,
    FilterSchema, Query)

...

@router.get("/venues/", response=list[VenueOut])
def venues(request, filters: VenueFilter = Query(...)):
    venues = Venue.objects.all()
    venues = filters.filter(venues)
    return venues
```

Add the Query type hint class to your list of imports.

Declare a query parameter, using the VenueFilter type, which takes a variable number of arguments.

The filters object is an instance of VenueFilter with a method that filters the venue's QuerySet based on the query parameters.

To see the new endpoint in action, you'll want some additional `Venue` objects: filtering when you only have one in the database isn't much fun. Use the Django Admin to add a couple more venues, and then visit http://localhost:8000/api/v1/bands/venues/?name=c. Your results will be dependent on what data you created, but it will be something like this:

```
[{
    "id": 1,
    "name": "CBGB",
    "description": "Take it CJ! 1-2-3-4",
    "slug": "cbgb-1",
    "url": "/api/v1/bands/venue/1/",
},
{
    "id": 8,
    "name": "Cherry Bar",
    "description": "Australia's best after party",
    "slug": "cherry-bar-8",
    "url": "/api/v1/bands/venue/8/",
}]
```

Don't forget to test it without a query parameter filter as well, to make sure you didn't break anything.